```
<img src="http://www.harmless.com/img" width="400" height="400" onmouseover="..." />
```

There are many other possibilities, like using a `<script>` tag to make a cross-site request to a URL with a JSONP or JavaScript response. The response is executable code that the attacker can find a way to run, possibly extracting sensitive data. To protect against this data leakage, we disallow cross-site `<script>` tags. Only Ajax requests may have JavaScript responses since `XMLHttpRequest` is subject to the browser Same-Origin policy - meaning only your site can initiate the request.

To protect against all other forged requests, we introduce a *required security token* that our site knows but other sites don't know. We include the security token in requests and verify it on the server. This is a one-liner in your application controller, and is the default for newly created rails applications:

```
protect_from_forgery with: :exception
```

This will automatically include a security token in all forms and Ajax requests generated by Rails. If the security token doesn't match what was expected, an exception will be thrown.

By default, Rails includes jQuery and an unobtrusive scripting adapter for jQuery, which adds a header called `X-CSRF-Token` on every non-GET Ajax call made by jQuery with the security token. Without this header, non-GET Ajax requests won't be accepted by Rails. When using another library to make Ajax calls, it is necessary to add the security token as a default header for Ajax calls in your library. To get the token, have a look at `<meta name='csrf-token' content='THE-TOKEN'>` tag printed by `<%= csrf_meta_tags %>` in your application view.

It is common to use persistent cookies to store user information, with `cookies.permanent` for example. In this case, the cookies will not be cleared and the out of the box CSRF protection will not be effective. If you are using a different cookie store than the session for this information, you must handle what to do with it yourself:

```
rescue_from ActionController::InvalidAuthenticityToken do |exception|
  sign_out_user # Example method that will destroy the user cookies
end
```

The above method can be placed in the `ApplicationController` and will be called when a CSRF token is not present or is incorrect on a non-GET request.

Note that *cross-site scripting (XSS) vulnerabilities bypass all CSRF protections*. XSS gives the attacker access to all elements on a page, so they can read the CSRF security token from a form or directly submit the form. Read more about XSS later.

4 Redirection and Files

Another class of security vulnerabilities surrounds the use of redirection and files in web applications.

4.1 Redirection

Redirection in a web application is an underestimated cracker tool: Not only can the attacker forward the user to a trap web site, they may also create a self-contained attack.

Whenever the user is allowed to pass (parts of) the URL for redirection, it is possibly vulnerable. The most obvious attack would be to redirect users to a fake web application which looks and feels exactly as the original one. This so-called phishing attack works by sending an unsuspicious link in an email to the users, injecting the link by XSS in the web application or putting the link into an external site. It is unsuspicious, because the link starts with the URL to the web application and the URL to the malicious site is hidden in the redirection parameter: http://www.example.com/site/redirect?to=www.attacker.com. Here is an example of a legacy action:

```
def legacy
  redirect_to(params.update(action:'main'))
end
```